

编译器设计实验-第三次实验

Haochen You

May 2026

1 实验任务（必做）

1.1 任务描述

构建给定文法的 LR(0) 项目集规范族，为后续生成 SLR(1) 分析表提供基础。

其中具体任务有以下七点：

1. 添加增广文法；
 2. 生成初始项目集；
 3. 计算闭包 (closure)；
 4. 计算转移 (Goto)；
 5. 结果生成（重复步骤 3-4，直到不再产生新项目集）；
 6. 输出所有项目集；
 7. 考虑是否属于 LR(0) 文法（无移进-归约或归约-归约冲突）。
- 另：实现的过程中我假设文件的输入都是合法的，具体输入格式和 PPT 给出的一致。

1.2 代码框架

数据结构：

表 1: 数据结构说明

数据结构名	含义
Production	表示一条文法产生式，包括左部非终结符和右部符号序列
Item	表示一个 LR(0) 项目，由产生式编号和圆点位置组成
ItemSet	表示一个 LR(0) 项目集，即规范族中的一个状态
Transition	表示项目集之间的 Goto 转移关系
Grammar	保存文法整体信息，包括开始符号、产生式、终结符和非终结符
LR0Result	保存 LR(0) 构造结果，包括项目集、转移关系和冲突信息

函数：

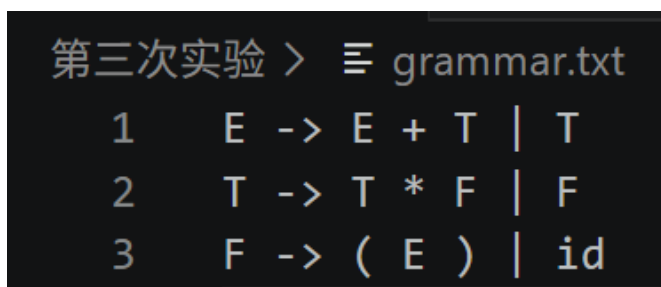
表 2: 主要函数说明

函数名	作用
<code>split_symbols()</code>	按空格切分产生式右部符号
<code>read_grammar_lines()</code>	从文件中读取文法规则
<code>parse_grammar()</code>	解析文法规则，生成产生式、终结符和非终结符集合
<code>augment_grammar()</code>	构造增广文法
<code>closure()</code>	计算 LR(0) 项目集的闭包
<code>go_to()</code>	计算项目集在某个文法符号下的 Goto 转移结果
<code>build_canonical_collection()</code>	构造 LR(0) 项目集规范族
<code>build_transitions()</code>	记录所有项目集之间的状态转移关系
<code>detect_conflicts()</code>	检查项目集中是否存在移进-归约或归约-归约冲突
<code>build_lr0()</code>	调用各步骤，完成 LR(0) 构造流程
<code>print_grammar_lines()</code>	输出原始文法规则
<code>print_result_skeleton()</code>	输出当前 LR(0) 构造结果框架

下面，我们来一步步讲如何实现。

1.3 Task0: 解析文法产生式

在开始构建 LR(0) 项目集之前，我们的代码先要能够解析文法产生式。为了之后拓展、测试的方便，我们全部使用文件 I/O。读入文件的格式如下：



```

第三次实验 > grammar.txt
1      E -> E + T | T
2      T -> T * F | F
3      F -> ( E ) | id
  
```

图 1: Input File

我们要把文法文件里的文本变成程序内部能处理的数据结构。比如上面形式的输入，程序要把它解析为多条产生式，也就是把 | 拆开，每个候选式都是一条独立的产生式。具体实现逻辑分为下面几步：

1. 按 -> 切分产生式左部和右部;
2. 第一条产生式左部作为开始符号
3. 按 | 拆分候选式
4. 每个候选式保存为一条 Production

1.4 Task1: 实现增广文法

这一步十分简单，添加增广产生式 $S' \rightarrow S$ ，并放在产生式数组第 0 位即可。代码如下：

```

1 Grammar augment_grammar(const Grammar &grammar)
2 {
3     Grammar res = grammar;
4     Production prod;
5     prod.left = res.augmented_start_symbol;
6     prod.right.push_back(res.start_symbol);
7     res.productions.insert(res.productions.begin(), prod);
8     res.nonterminals.insert(res.augmented_start_symbol);
9     return res;
10 }

```

1.5 Task2: 生成初始项目集

那么初始项目就是在第 0 条产生式最前面加上圆点，变成 $E' \rightarrow \cdot E$ 。这个圆点表示语法分析刚开始，还没有读入任何符号，下一步要识别的是 E 。在程序中，不需要真的把圆点写进字符串里，而是用产生式编号和圆点位置来表示这个项目，例如 `production_index = 0` 表示使用第 0 条产生式，`dot_pos = 0` 表示圆点在最前面。这样就可以得到初始项目集 I_0 。后面实现闭包操作后，再根据这个项目继续补充。代码如下：

```

1 vector<ItemSet> build_canonical_collection(const Grammar &grammar)
2 {
3     Item item;
4     item.production_index = 0;
5     item.dot_pos = 0;
6     ItemSet item_set;
7     item_set.id = 0;
8     item_set.items.push_back(item);
9     vector<ItemSet> item_sets;
10    item_sets.push_back(item_set);
11    return item_sets;
12 }

```

1.6 Task3: 计算闭包 (closure)

下一步我们计算闭包。它的作用是：如果某个项目的圆点后面是一个非终结符，就要把这个非终结符对应的所有产生式也加入项目集中，并且这些新加入的产生式圆点都放在最前面。加入之后，还要继续检查新项目，再继续检查，直到没有新的项目可以加入为止。代码如下：

```

1 vector<Item> closure(const Grammar &grammar, const vector<Item> &seed)
2 {
3     vector<Item> res = seed;
4     for (int i = 0; i < (int)res.size(); i++)
5     {
6         Item item = res[i];
7         Production prod = grammar.productions[item.production_index];
8         if (item.dot_pos >= (int)prod.right.size()) continue;
9         string symbol = prod.right[item.dot_pos];
10        if (grammar.nonterminals.count(symbol) == 0) continue;
11        for (int j = 0; j < (int)grammar.productions.size(); j++)
12        {
13            if (grammar.productions[j].left != symbol) continue;
14            Item new_item;
15            new_item.production_index = j;
16            new_item.dot_pos = 0;
17            bool existed = false;
18            for (int k = 0; k < (int)res.size(); k++)
19            {
20                if (res[k].production_index == new_item.production_index &&

```

```

21         res[k].dot_pos == new_item.dot_pos)
22     {
23         existed = true;
24         break;
25     }
26 }
27 if (!existed) res.push_back(new_item);
28 }
29 }
30 return res;
31 }

```

当然，在 Task2 中也要做一定的修改，把原来存入的变成闭包形式即可。

1.7 Task4: 计算转移 (Goto)

简单说，Goto 的作用是：从一个项目集 I 出发，遇到某个符号 X 后，看看哪些项目的圆点后面正好是 X，然后把把这些项目的圆点向右移动一位，形成一个新的项目集，最后再对这个新项目集求闭包。实现如下：

```

1 vector<Item> go_to(const Grammar &grammar, const vector<Item> &items, const string &symbol)
2 {
3     vector<Item> moved;
4     for (int i = 0; i < (int)items.size(); i++)
5     {
6         Item item = items[i];
7         Production prod = grammar productions[item.production_index];
8
9         if (item.dot_pos >= (int)prod.right.size())
10             continue;
11         if (prod.right[item.dot_pos] == symbol)
12         {
13             Item new_item;
14             new_item.production_index = item.production_index;
15             new_item.dot_pos = item.dot_pos + 1;
16             moved.push_back(new_item);
17         }
18     }
19     return closure(grammar, moved);
20 }

```

其实现逻辑是：遍历项目集中的每一个项目，找到那些“圆点后面的符号正好是 X”的项目，然后把把这些项目的圆点向右移动一位，形成一个新的项目集合。移动完成后，还要对这个新集合调用 closure()，补全可能出现的新项目，最后返回这个闭包结果。如果没有任何项目能通过符号 X 转移，那么返回的就是空集合。

1.8 Task5: 结果生成

这一步要实现的是完整的 LR(0) 项目集规范族构造，也就是把前面已经写好的 closure() 和 go_to() 串起来，不断生成新的项目集。在 build_canonical_collection 中增添的逻辑如下：

程序先生成初始项目集 I0，然后依次检查已有的每一个项目集，对所有可能的文法符号计算 Goto(I, X)。

如果计算结果为空，就说明这个符号不能产生转移，跳过；如果计算结果不是空集，就把它和已经存在的项目集进行比较。如果之前已经出现过这个项目集，就不用重复添加；如果是新的项目集，就给它分配新的编号，加入项目集数组。

直到所有项目集都检查完，没有新的项目集再产生，就构造完成了。

1.9 Task6: 输出所有项目集

这一步实现输出即可。我们遍历所有项目集，对每个项目集里的每个项目，找到它对应的产生式。之后依次输出其左部、右部。只需注意输出右部时，根据 dot_pos 插入圆点即可。代码不在此列出。

针对测试的 Input，代码输出如下：

```
PS E:\2026_Junior_S2\CompilerLab\Lab3> .\lr0.exe
LR(0)项目集规范族构造程序
文法文件: grammar.txt

原始文法
E -> E + T | T
T -> T * F | F
F -> ( E ) | id

文法解析结果
开始符号: E
增广开始符号: E'
非终结符: E E' F T
终结符: ( ) * + id
产生式:
0: E' -> E
1: E -> E + T
2: E -> T
3: T -> T * F
4: T -> F
5: F -> ( E )
6: F -> id

增广文法
E' -> E

LR(0)项目集
I0:
E' -> . E
E -> . E + T
E -> . T
T -> . T * F
T -> . F
F -> . ( E )
F -> . id

I1:
E' -> E .
E -> E . + T

I2:
T -> F .
```

(a) 程序输出前半部分

```
PS E:\2026_Junior_S2\CompilerLab\Lab3> .\lr0.exe
I3:
E -> T .
T -> T . * F

I4:
F -> ( . E )
E -> . E + T
E -> . T
T -> . T * F
T -> . F
F -> . ( E )
F -> . id

I5:
F -> id .

I6:
E -> E + . T
T -> . T * F
T -> . F
F -> . ( E )
F -> . id

I7:
T -> T * . F
F -> . ( E )
F -> . id

I8:
F -> ( E . )
E -> E . + T

I9:
E -> E + T .
T -> T . * F

I10:
T -> T * F .

I11:
F -> ( E ) .

Goto转移
TODO: 记录Goto转移关系

冲突检查
TODO: 检查LR(0)冲突
```

(b) 程序输出后半部分

图 2: LR(0) 项目集规范族构造程序运行结果

1.10 Task7: 移进-归约/归约-归约冲突检查

接下来我们进行最后一部分，移进-归约或归约-归约冲突检查。

程序依次遍历每一个项目集，统计这个项目集中有没有“归约项目”和“移进项目”。具体概念不再赘述。程序会把发现冲突的项目集编号记录下来，最后统一输出。可以发现，能够成功检测出来冲突。

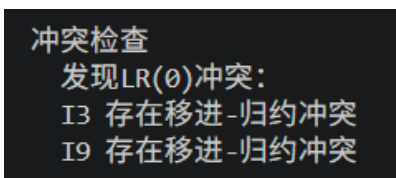


图 3: 冲突检查结果

2 实验任务（选做）

2.1 任务描述

实现给定文法的 LR(0) 项目集规范族可视化。

2.2 成果展示

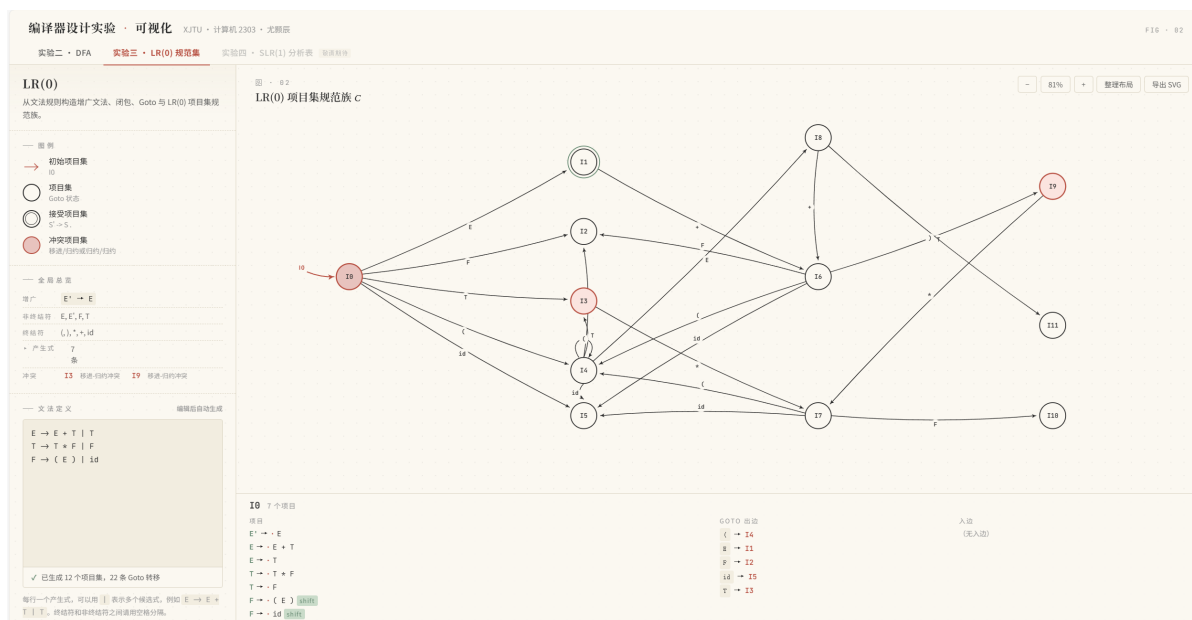


图 4: 可视化

代码仓库: <https://github.com/Yukino-chyan/CompilerWeb>。

3 个人总结

本次实验让我将语法分析中理论转化为具体的 C++ 实现。在基础功能上，我完成了对增广文法的闭包 $\text{closure}(I)$ 与 $\text{Goto}(I, X)$ 的计算，构建出完整的项目集规范族 C 及其状态转移关系，并实现了对移进-归约、归约-归约冲突的判定。在此基础上，我进一步搭建了可视化前端，支持画布缩放、平移、节点拖拽等功能，以及项目集详情的查询。